CSE3300 Programming Assignment 1

WORDQ: a TCP Word-Query Client and Multi-threaded Server

Design Document (multi-threaded version)

Colin Willour | NetID: cjw22003

1. Description

1.1 What the program does

A client sends a wildcard query typed at the keyboard to a server over TCP. The server holds an English word list (`wordlist.txt`, 69,903 words) and returns every word that **contains** the query as a substring, where each ? stands for exactly one arbitrary character. For example `a?t` is treated as `*a?t*` and matches ant, anticipate, mantis, rant and 4,256 other words. The client prints the status, the number of matching words and each word.

There are two versions. The **basic** version is a single-threaded server serving one client at a time, plus a client that sends one query and exits. The **multi-threaded** version is a server that starts one thread per client so several clients are served at once, plus a client that keeps one connection open and sends many queries until the user types `quit`.

1.2 Files

File	Purpose
`wordq_protocol.py`	Shared module: protocol constants, word loading, wildcard matching, request processing (server side), `serve_client()` loop, and client helpers `connect()`, `send_line()`, `read_response()`.
`basic_server.py`	Single-threaded server; `listen(1)`, calls `serve_client()` directly, so other clients wait.
`basic_client.py`	Sends ONE query, prints the result, exits.
`mt_server.py`	Multi-threaded server; `dispatcher()` starts a `handle_client` thread per connection (structure from the provided thread-server.py).
`mt_client.py`	Sends many queries on one persistent connection until `quit` (or EOF).
`test_raw_protocol.py`	Test aid: sends malformed lines (empty QUERY, unknown command, 2000-char line) the normal clients never send.
`wordlist.txt`	The word list, one word per line (69,903 words).
`logs/`	Captured output of every test run in Section 4.

1.3 How to run

Python 3, standard library only. Run each program in its own terminal from the `pa1/` folder. Servers take `[port] [wordlist_path]` (defaults 12000 and `wordlist.txt` next to the script); clients take `[host] [port]` (defaults 127.0.0.1 and 12000).

```
python3 mt_server.py                # terminal 1 (or basic_server.py)
python3 mt_client.py                # terminal 2 (or basic_client.py)
python3 mt_client.py                # terminal 3: a second simultaneous client
python3 mt_server.py 13000 other.txt # custom port and word list
python3 basic_client.py 10.0.0.5 13000
```

1.4 Application protocol (WORDQ)

WORDQ is a text, line-oriented request/response protocol over TCP, modelled on HTTP and SMTP: the client sends a command line, the server answers with a status line containing a numeric code, a reason phrase and the number of result lines that follow.

- **Transport:** TCP, default port 12000. **Encoding:** UTF-8. Every line ends with `\n` (a trailing `\r` is tolerated).
- **Request line limit:** 1024 bytes including the newline.
- **Commands** (keyword is case-insensitive): `QUERY <pattern>` and `QUIT`.
- **Response:** status line `<code> <reason> <count>\n`, then exactly `<count>` lines of one word each. Because TCP is a byte stream with no message boundaries, the count is the framing: the client knows exactly how many lines belong to the reply.

Status line	Meaning	Connection afterwards
`200 OK <n>`	n >= 1 matching words follow, one per line, in word-list order	stays open
`200 BYE 0`	reply to QUIT	server closes
`404 NOT_FOUND 0`	valid query, no word matched	stays open
`400 BAD_REQUEST 0`	empty pattern, unknown command, invalid UTF-8	stays open
`400 BAD_REQUEST 0`	request line longer than 1024 bytes	server closes (it cannot find where the next line starts)

Matching rule. The pattern is converted to a regular expression: `?` becomes `.` and every other character is passed through `re.escape()`, so `(`, `)` and `-` are literal. The regex is compiled with `re.IGNORECASE | re.DOTALL` and tested with `re.search()` (no anchors), which gives the `*pattern*` substring semantics. `?` matches any single character, including punctuation such as `-` or `(`. This exact rule reproduces the required counts (24071, 69903, 414, 13).

Example session (C = client, S = server), taken from the real test run:

```
C: QUERY -?-
S: 200 OK 13
S: bric-a-brac
S: cap-a-pie
S:   ... 10 more words ...
S: vis-a-vis
C: query zzzzqx
S: 404 NOT_FOUND 0
C: FIND a?t
S: 400 BAD_REQUEST 0
C: QUIT
S: 200 BYE 0
   (server closes the connection)
```

1.5 How the server works

- At start-up the server reads the word list once into a Python list (`load_words`), creates an `AF_INET/SOCK_STREAM` socket, sets `SO_REUSEADDR` so it can be restarted immediately, and binds to `("", port)`, i.e. all interfaces. A bind error (port in use) prints a message and exits; Ctrl-C shuts the server down cleanly.
- **Multi-threaded server:** `listen(5)`; the main thread runs `dispatcher()`, which loops on `accept()` and calls `thread.start_new_thread(handle_client, (conn, addr, words))` for every new connection, then immediately returns to `accept()`. `handle_client` wraps `serve_client()` in try/except/finally and prints `Client <addr> finished at <time>`. All threads share one word list; it is never modified after loading, so no lock is needed.
- **Basic server:** `listen(1)`; calls `serve_client()` directly in the main thread, so a second client is only served once the first one has finished.
- `serve_client()` (shared): wraps the socket in a buffered reader (`conn.makefile("rb")`), reads one line with `readline(1024)`, passes it to `process_request()`, which returns the response bytes and whether to keep the connection open, and sends them with `sendall()`. It stops when the client sends QUIT, sends an over-long line, or closes the connection, and always closes the socket in `finally`. Before closing after a 400 for an over-long line, it shuts down its sending side and discards the rest of the client's input; otherwise the kernel would send a TCP reset, which can destroy the 400 reply before the client reads it.

1.6 How the client works

- `connect()` uses `socket.create_connection()` with a 10 s connect timeout, then sets a 30 s read timeout so a dead server cannot hang the client forever. A refused connection prints a friendly message and exits with status 1.
- `read_response()` reads the status line, validates it (three fields, numeric code and count), then reads exactly `count` word lines. A malformed status line or a reply cut off early raises `ProtocolError`.
- **basic_client.py** asks `Enter query:` once (empty input is rejected locally), sends `QUERY <q>`, prints `Server replied: <code> <reason>`, `Number of matching words: N` and the words, then exits.

- **mt_client.py** connects once and loops on `Enter query (or 'quit'):`. Empty lines are ignored; `quit` (or EOF/Ctrl-D) sends QUIT, reads `200 BYE` and exits. Network/protocol errors and Ctrl-C are handled and the socket is always closed.

1.7 Borrowed code vs. code written by me

`mt_server.py` is adapted from the provided sample `thread-server.py` (Mark Lutz, "Programming Python", O'Reilly). Borrowed lines are marked `[BORROWED]` in the source.

Borrowed from thread-server.py	Written for this assignment
Dispatcher / per-client handler structure: `dispatcher()` loops on `accept()` and starts `thread.start_new_thread(handler, (conn, ...))`	The WORDQ protocol, status codes and count-based framing (`wordq_protocol.py`)
`now()` helper: `time.ctime(time.time())`	Wildcard matching (`compile_pattern`, `find_matches`)
Listen backlog of 5	Word list loaded once and shared read-only by all threads
Use of the low-level `_thread` module (`import _thread as thread`)	Error handling: try/except/finally in `handle_client`, graceful close, `OSError` handling
	Command-line arguments, `SO_REUSEADDR`, bind-error and Ctrl-C handling; all client code; basic server

Removed from the sample: `time.sleep(5)` (simulated blocking work) and the `Echo=>` reply loop, which was replaced by `wq.serve_client()`. The sample also bound to `localhost` port 2222; this version binds all interfaces on port 12000.

2. Tradeoffs

Decision	Choice made	Alternative and why it was not chosen
Message format	Text, line-oriented, UTF-8	Binary (length-prefixed) would be more compact, but text is human-readable, testable with telnet/nc, and words are already text; the overhead (one newline per word) is small.
Framing the reply	Word count in the status line	A terminator line (e.g. `.` as in SMTP) needs escaping if a word equals the terminator. With the count, the client knows the size in advance and detects truncation.
Connection lifetime	Persistent: many queries per connection (mt_client)	One connection per query (HTTP/1.0 style) is simpler but pays a TCP handshake per query. QUIT gives a clean, explicit end.
Concurrency model	One thread per client	A fixed thread pool bounds resource use under many clients, but thread-per-client is simpler, matches the provided sample, and is fine for a class-sized load. Idle clients each hold a thread.
Shared data	Word list read-only, shared by all threads, no locks	Copying per thread wastes memory; a lock would only be needed if the list could change (see RELOAD in Section 3).
Search method	Linear regex scan of all words per query	An index (e.g. by length or n-grams) would be faster, but a scan of 70k words takes milliseconds, is obviously correct, and handles `?` anywhere.

Decision	Choice made	Alternative and why it was not chosen
Meaning of ?	Any single character, including punctuation	Letters-only (`[a-z]`) is closer to the handout wording, but the required test counts (e.g. ? = 69,903, which includes 1-character words) are only reproduced with "any character".
Bad requests	400 and keep the connection open; close only for an over-long line	Closing on every error is simpler but unfriendly. After an over-long line the server cannot tell where the next request starts, so it closes.
Bind address	`""` (all interfaces)	`localhost` is safer but only reachable from the same machine; all interfaces lets clients on other hosts connect.
Thread module	Low-level `_thread`	`threading.Thread` is the modern API (daemon flags, join, names). `_thread` was kept to stay close to the provided sample; behaviour is the same here.

3. Extensions

- **Thread pool and timeouts.** Replace thread-per-client with `concurrent.futures.ThreadPoolExecutor(max_workers=N)` and `executor.submit(handle_client, conn, addr, words)` in the dispatcher; set `conn.settimeout(300)` so idle clients are disconnected and their threads freed.
- **Paging (LIMIT/OFFSET).** Add `QUERY <pattern> LIMIT <n> OFFSET <k>`; the server returns `200 OK <sent> <total>` with a slice of the result, so a client can show ? (69,903 words) page by page.
- **COUNT and HELP commands.** `COUNT <pattern>` returns `200 OK 0 <n>`-style counts without the words (cheap for large results); `HELP` returns the list of supported commands as body lines. Both only need new branches in `process_request()`.
- **Caching.** Keep a small LRU cache (`functools.lru_cache` on a function returning a tuple of matches, keyed on the lower-cased pattern) so repeated popular queries skip the scan. Thread-safe because the list is read-only.
- **RELOAD with a lock.** A `RELOAD` command (or a signal) re-reads the word list. Build the new list fully, then swap a shared reference under a `threading.Lock` (readers take the lock only to grab the current reference), and clear the cache.
- **TLS.** Wrap sockets with `ssl.SSLContext.wrap_socket()` (server certificate on the server side, `create_default_context()` on the client) so queries and results are encrypted.
- **UDP version.** One datagram per request; the reply carries the count and words, split across numbered datagrams if it exceeds the datagram size. The multi-threaded UDP server needs a lock around `sendto` or a single receiving thread handing work to workers.

4. Test cases

All outputs below were captured from real runs on Linux, Python 3, with server and clients on 127.0.0.1:12000. Full outputs, including every matching word, are saved in `pa1/logs/`. Long word lists are shortened here to their first and last lines. Red placeholders mark where terminal screenshots go. Keyboard input was piped in for the captures, so the typed text is shown in the `$` header line and the reply appears on the same line as the prompt.

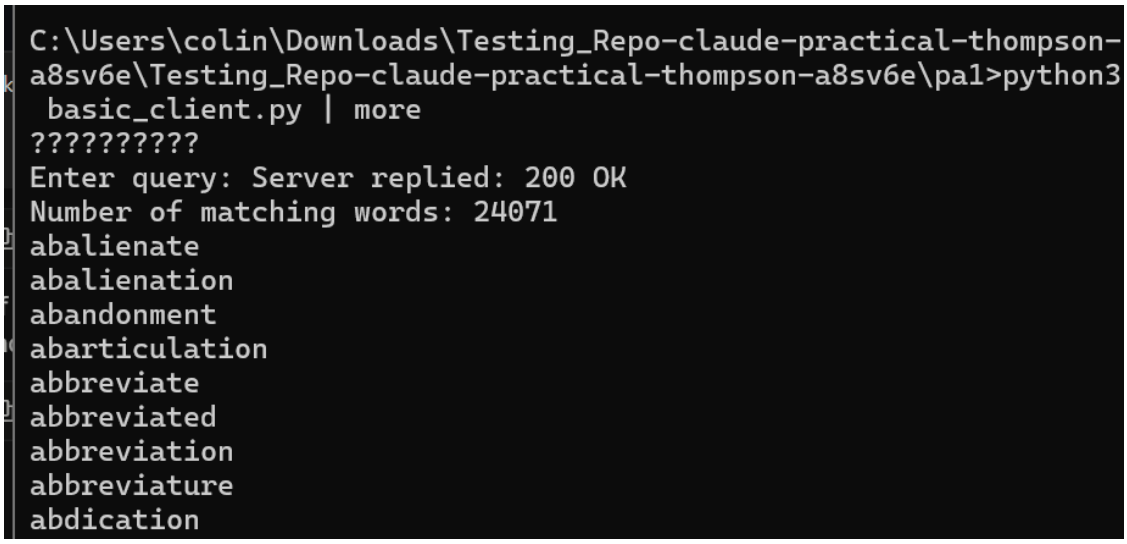
4.1 Summary

#	Test	Expected	Observed	Result
1	Required: `??????????`	24071 words	200 OK, 24071	Pass
2	Required: `?`	69903 words	200 OK, 69903	Pass
3	Required: `?(a)`	414 words	200 OK, 414	Pass
4	Required: `-?-`	13 words	200 OK, 13	Pass
5	Handout example `a?t`	ant, mantis, …	200 OK, 4260	Pass
6	No match `zzzzqx`	404	404 NOT_FOUND, 0	Pass
7	Empty query in client	rejected locally	message, exit 1	Pass
8	Server not running	friendly error	message, exit 1	Pass
9	mt_client: 6 queries + quit on one connection	all answered, BYE	as expected	Pass
10	mt_client: EOF (Ctrl-D)	treated as quit	200 BYE	Pass
11	Empty `QUERY` (raw)	400	400 BAD_REQUEST	Pass
12	Unknown command `FIND`	400, stays open	400; next query 200 OK	Pass
13	Lower-case `query`	accepted	404 NOT_FOUND	Pass
14	2000-character line	400, then close	400; connection closed	Pass
15	Two clients at once, mt_server	B served while A idle	B done 5 s before A	Pass
16	Two clients, basic_server	B waits for A	B answered only after A quit	Pass
17	Port already in use	bind error message	message, exit	Pass

4.2 Required queries (basic server and basic client)

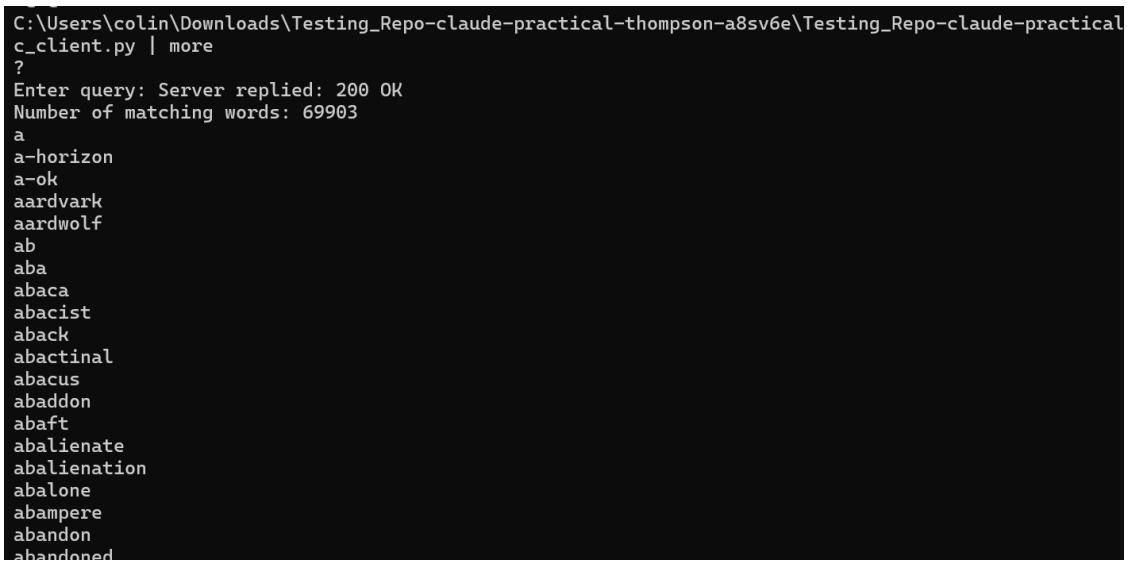
Test 1: ??????????

```
$ python3 basic_client.py      (typed: ??????????)
Enter query: Server replied: 200 OK
Number of matching words: 24071
abalienate
abalienation
abandonment
abarticulation
abbreviate
   ... (24065 more lines omitted; full output in logs/) ...
zygophyllum
[exit code 0]
```



Test 2: ?

```
$ python3 basic_client.py      (typed: ?)
Enter query: Server replied: 200 OK
Number of matching words: 69903
a
a-horizon
a-ok
aardvark
aardwolf
   ... (69897 more lines omitted; full output in logs/) ...
zymurgy
[exit code 0]
```



Test 3: ?(a)

```
$ python3 basic_client.py      (typed: ?(a))
Enter query: Server replied: 200 OK
Number of matching words: 414
acting(a)
activating(a)
admitted(a)
advance(a)
advanced(a)
   ... (408 more lines omitted; full output in logs/) ...
zero(a)
[exit code 0]
```

```
C:\Users\colin\Downloads\Testing_Repo-claude-practical-thompson-a8sv6e\Testing_
Repo-claude-practical-thompson-a8sv6e\pa1>python3 basic_client.py | more
?(a)
Enter query: Server replied: 200 OK
Number of matching words: 414
acting(a)
activating(a)
admitted(a)
advance(a)
advanced(a)
aforesaid(a)
aft(a)
after(a)
after-school(a)
afternoon(a)
aged(a)
agreeing(a)
```

Test 4: -?-

```
$ python3 basic_client.py      (typed: -?-)
Enter query: Server replied: 200 OK
Number of matching words: 13
bric-a-brac
cap-a-pie
cock-a-doodle-doo
cock-a-leekie
high-muck-a-muck
   ... (7 more lines omitted; full output in logs/) ...
vis-a-vis
[exit code 0]
```

```
C:\Users\colin\Downloads\Testing_Repo-claude-practical-thompson-a8sv6e\Testing_
Repo-claude-practical-thompson-a8sv6e\pa1>python3 basic_client.py | more
-?-
Enter query: Server replied: 200 OK
Number of matching words: 13
bric-a-brac
cap-a-pie
cock-a-doodle-doo
cock-a-leekie
high-muck-a-muck
love-in-a-mist
man-on-a-horse
pied-a-terre
pit-a-pat
rat-a-tat-tat
rub-a-dub
tete-a-tete
vis-a-vis
```

Test 5: a?t (handout example)

```
$ python3 basic_client.py      (typed: a?t)
Enter query: Server replied: 200 OK
Number of matching words: 4260
abactinal
abaft
abarticulation
abattis
abattoir
   ... (4254 more lines omitted; full output in logs/) ...
zygodactyl
[exit code 0]
```

```
C:\Users\colin\Downloads\Testing_Repo-claude-practical-thompson-a8sv
Repo-claude-practical-thompson-a8sv6e\pa1>python3 basic_client.py |
a?t
Enter query: Server replied: 200 OK
Number of matching words: 4260
abactinal
abaft
abarticulation
abattis
abattoir
abdicant
aberrant
abeyant
ablactation
ablaut
abreast
absonant
abstract
abstracted
abstractedly
abstractedness
abstraction
abstractionism
abstractionist
abstractive
abstractly
abstractness
abstractor
```

Test 6: zzzzqx (no match)

```
$ python3 basic_client.py      (typed: zzzzqx)
Enter query: Server replied: 404 NOT_FOUND
Number of matching words: 0
[exit code 0]
```

[SCREENSHOT PLACEHOLDER: Test 6: zzzzqx (no match) (basic_client terminal)]

Test 7: empty query

```
$ python3 basic_client.py      (typed: )
Enter query: Query must not be empty.
[exit code 1]
```

```
C:\Users\colin\Downloads\Testing_Repo-claude-practical-thompson-a8sv6e\Testing_
Repo-claude-practical-thompson-a8sv6e\pa1>python3 basic_client.py | more

Enter query: Query must not be empty.
```

Test 8: server not running

```
Enter query: Could not connect to 127.0.0.1:12000 ([Errno 111] Connection refused). Is
the server running?
[exit code 1]
```

[SCREENSHOT PLACEHOLDER: Test 8: server not running (basic_client terminal)]

Server side of tests 1-6 (`logs/basic_server.log`):

```
Loaded 69903 words from /home/user/Testing_Repo/pa1/wordlist.txt
Basic WORDQ server listening on port 12000 (Ctrl-C to stop)
Connected by ('127.0.0.1', 46484)
('127.0.0.1', 46484)  b'QUERY ??????????\n' -> 200 OK 24071
Client ('127.0.0.1', 46484) closed the connection
Finished with ('127.0.0.1', 46484)
Connected by ('127.0.0.1', 46486)
('127.0.0.1', 46486)  b'QUERY ?\n' -> 200 OK 69903
Client ('127.0.0.1', 46486) closed the connection
Finished with ('127.0.0.1', 46486)
Connected by ('127.0.0.1', 46498)
('127.0.0.1', 46498)  b'QUERY ?(a)\n' -> 200 OK 414
Client ('127.0.0.1', 46498) closed the connection
Finished with ('127.0.0.1', 46498)
Connected by ('127.0.0.1', 46506)
('127.0.0.1', 46506)  b'QUERY -?-\n' -> 200 OK 13
Client ('127.0.0.1', 46506) closed the connection
Finished with ('127.0.0.1', 46506)
Connected by ('127.0.0.1', 46514)
('127.0.0.1', 46514)  b'QUERY a?t\n' -> 200 OK 4260
```

[SCREENSHOT PLACEHOLDER: basic_server terminal]

4.3 Multi-threaded client: several queries on one connection

Test 9 sends the four required queries plus `a?t`, `zzzzqx`, an empty line (ignored) and `quit` on one connection (`logs/mt_client_session.txt`). Status lines from that run:

```
$ python3 mt_client.py   (typed one per line: ??????????, ?, ?(a), -?-, a?t, zzzzqx,
<empty>, quit)
Connected to 127.0.0.1:12000
Enter query (or 'quit'): Server replied: 200 OK
Number of matching words: 24071
Enter query (or 'quit'): Server replied: 200 OK
Number of matching words: 69903
Enter query (or 'quit'): Server replied: 200 OK
Number of matching words: 414
Enter query (or 'quit'): Server replied: 200 OK
Number of matching words: 13
Enter query (or 'quit'): Server replied: 200 OK
Number of matching words: 4260
Enter query (or 'quit'): Server replied: 404 NOT_FOUND
Number of matching words: 0
Enter query (or 'quit'): Server replied: 200 BYE
```

Server log for this session (`logs/mt_server.log`):

```
Loaded 69903 words from /home/user/Testing_Repo/pa1/wordlist.txt
Multi-threaded WORDQ server listening on port 12000 (Ctrl-C to stop)
Server connected by ('127.0.0.1', 34836) at Fri Oct 16 00:14:31 2026
('127.0.0.1', 34836)  b'QUERY ??????????\n' -> 200 OK 24071
('127.0.0.1', 34836)  b'QUERY ?\n' -> 200 OK 69903
('127.0.0.1', 34836)  b'QUERY ?(a)\n' -> 200 OK 414
('127.0.0.1', 34836)  b'QUERY -?-\n' -> 200 OK 13
('127.0.0.1', 34836)  b'QUERY a?t\n' -> 200 OK 4260
('127.0.0.1', 34836)  b'QUERY zzzzqx\n' -> 404 NOT_FOUND 0
('127.0.0.1', 34836)  b'QUIT\n' -> 200 BYE 0
Client ('127.0.0.1', 34836) finished at Fri Oct 16 00:14:32 2026
```

[SCREENSHOT PLACEHOLDER: mt_client session with the four required queries]

Test 10, EOF counts as quit (`logs/mt_client_eof.txt`):

```
$ python3 mt_client.py   (typed: -?- then EOF/Ctrl-D)
Connected to 127.0.0.1:12000
Enter query (or 'quit'): Server replied: 200 OK
Number of matching words: 13
bric-a-brac
   ... (11 more lines omitted; full output in logs/) ...
vis-a-vis
Enter query (or 'quit'):
Server replied: 200 BYE
[exit code 0]
```

4.4 Malformed requests

Tests 11-14 use `test_raw_protocol.py`, which sends raw lines over one connection (`logs/raw_protocol_tests.txt`):

```
--- Test: empty QUERY
SEND b'QUERY\n'
RECV 400 BAD_REQUEST 0
SEND b'QUERY    \n'
RECV 400 BAD_REQUEST 0
--- Test: unknown command, then check connection stays open
SEND b'FIND a?t\n'
RECV 400 BAD_REQUEST 0
SEND b'QUERY -?-\n'
RECV 200 OK 13
connection still open: True
--- Test: lower-case command (commands are case-insensitive)
SEND b'query zzzzqx\n'
RECV 404 NOT_FOUND 0
--- Test: 2000-character line
SEND b'QUERY aaaaaaaaaaaaa...(2007 bytes)'
RECV 400 BAD_REQUEST 0
server closed connection: True
```

Matching server log lines:

```
Server connected by ('127.0.0.1', 58870) at Fri Oct 16 00:14:38 2026
('127.0.0.1', 58870)  b'QUERY\n' -> 400 BAD_REQUEST 0
('127.0.0.1', 58870)  b'QUERY    \n' -> 400 BAD_REQUEST 0
('127.0.0.1', 58870)  b'FIND a?t\n' -> 400 BAD_REQUEST 0
('127.0.0.1', 58870)  b'QUERY -?-\n' -> 200 OK 13
('127.0.0.1', 58870)  b'query zzzzqx\n' -> 404 NOT_FOUND 0
('127.0.0.1', 58870)  b'QUERY aaaaaaaaaaaaaaaaaaaaaaaaa...' -> 400 BAD_REQUEST 0
Client ('127.0.0.1', 58870) finished at Fri Oct 16 00:14:38 2026
```

[SCREENSHOT PLACEHOLDER: test_raw_protocol.py output]

4.5 Two clients at the same time

Test 15, multi-threaded server. Client A connects, queries `-?-`, then stays idle for 6 s. One second later client B connects, queries `?(a)` and `zzzzqx` and quits, all while A is still connected (`logs/mt_two_clients.txt`):

```
[00:14:32] client A connects to MT server, queries -?-, then stays idle 6 s
[00:14:33] client B connects while A is still connected
[00:14:33] client B finished (A still connected)
[00:14:38] client A finished
```

The server log shows the two connections overlapping: B (port 34860) is accepted and served while A (port 34852) is still open, and A finishes last:

```
Server connected by ('127.0.0.1', 34852) at Fri Oct 16 00:14:32 2026
('127.0.0.1', 34852)  b'QUERY -?-\n' -> 200 OK 13
Server connected by ('127.0.0.1', 34860) at Fri Oct 16 00:14:33 2026
('127.0.0.1', 34860)  b'QUERY ?(a)\n' -> 200 OK 414
('127.0.0.1', 34860)  b'QUERY zzzzqx\n' -> 404 NOT_FOUND 0
('127.0.0.1', 34860)  b'QUIT\n' -> 200 BYE 0
Client ('127.0.0.1', 34860) finished at Fri Oct 16 00:14:33 2026
('127.0.0.1', 34852)  b'QUIT\n' -> 200 BYE 0
Client ('127.0.0.1', 34852) finished at Fri Oct 16 00:14:38 2026
```

[SCREENSHOT PLACEHOLDER: three terminals: mt_server, client A idle, client B served]

Test 16, basic server. Same scenario against `basic_server.py`: B connects at 00:14:25 but gets no reply until A quits at 00:14:30, because the single-threaded server only accepts B once it has finished with A (`logs/basic_server_second_client_waits.txt`):

```
[00:14:24] client A (mt_client) connects to BASIC server and stays idle 6 s
[00:14:25] client B (basic_client) starts, query zzzzqx
[00:14:30] client A finished (sent quit)
[B] Enter query: Server replied: 404 NOT_FOUND
[B] Number of matching words: 0
[00:14:30] client B finished
```

[SCREENSHOT PLACEHOLDER: basic_server: second client waiting]

Test 17, port already in use (second mt_server started while the first is running):

```
Loaded 69903 words from /home/user/Testing_Repo/pa1/wordlist.txt
Cannot bind to port 12000: [Errno 98] Address already in use
```

4.6 Known limitations

- Leading and trailing spaces in a query are stripped, so a pattern cannot start or end with a space (no word in the list contains a space).
- Every client gets its own thread with no upper limit, and idle clients are never timed out, so a very large number of connections could exhaust server threads (see the thread-pool extension).
- A query such as ? returns the whole list (about 650 KB) in one reply; there is no paging.
- The word list must be UTF-8; it is loaded once, so changes to the file need a server restart.

5. Use of AI

All of the code (the four programs, `wordq_protocol.py` and `test_raw_protocol.py`) and this design document were created with Claude (Anthropic's AI assistant), from the assignment handout and the provided `thread-server.py`. There is no separate pre-AI version of the code: AI was used to create the code, not to refine an existing one, so no "before refinement" version or list of refinement suggestions exists. The borrowed parts of the sample are identified in Section 1.7 and marked in the source.